

On the Automatic Identification of Misconfiguration Errors in Cloud Native Systems

Brian S. Mitchell
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
bmitchell@drexel.edu

Spiros Mancoridis
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
mancors@drexel.edu

Jainam Kashyap
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
jk3862@drexel.edu

Abstract

The frequency of notable data breaches in cloud native systems has increased over the past several years causing many problems for both large and small organizations. These systems have a very large attack surface associated with the use of many publicly-accessible APIs that can make differentiating between normal and malicious behavior difficult to classify. The nature of managing every aspect of cloud native systems is prone to misconfiguration errors, requiring the setting and auditing of thousands of parameters in enterprise systems. Successful data breaches require bad actors to remain undetected for long periods of time so that data exfiltration does not trigger intrusion protection controls. In this paper we recreate a realistic cloud native environment to emulate a data breach attack using common API misconfiguration mistakes. We then introduce a tool we created to collect system call data from the kernels in a Kubernetes cluster that is used to train machine learning models capable of differentiating normal from suspicious activity during emulated stealthy attacks.

CCS Concepts

• Security and privacy → Software and application security; • Computer systems organization → Cloud computing; • Computing methodologies → Anomaly detection.

Keywords

Cloud Native Architectures, Cloud Security, Machine Learning, Behavioral Anomaly Detection

ACM Reference Format:

Brian S. Mitchell, Spiros Mancoridis, and Jainam Kashyap. 2024. On the Automatic Identification of Misconfiguration Errors in Cloud Native Systems. In *2024 the 7th Artificial Intelligence and Cloud Computing Conference (AICCC) (AICCC 2024)*, December 14–16, 2024, Tokyo, Japan. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3719384.3719463>

1 Introduction

In recent years, one of the biggest security challenges facing cloud native systems is related to the detection and prevention of data breaches. A data breach happens when a user gains access to unauthorized data from a system. These problems can happen for many reasons, but are often the result of compounding many simple errors

associated with the management of the large number of technical assets and configuration settings in these types of systems. When exploited by bad actors, data breaches can result in significant legal, financial and reputation consequences for organizations and their customers.

Data breaches are attractive to malicious actors because of the direct- and indirect-value of the exfiltrated data. IBM's 2023 report on the cost of a data breach [18] estimates that the global average cost of a data breach has risen to \$4.45M USD in 2023, with the value of a specific individual's data record being assessed at \$165 USD. The United States has the highest overall data breach cost, averaging \$9.48M per incident. Besides using the vast amounts of stolen data for direct profit by selling it on the dark web, these types of attacks can also be used for extortion or ransom if the stolen data targets executives or intellectual property.

Like many cybersecurity attacks, data breaches require strategies for gaining initial access to a system, and then for covertly exporting data once inside a breached system. According to the CrowdStrike 2024 Global Threat Report [10] adversaries continue to reduce their reliance on using malware to break into systems to obtain initial access. Specifically, malware-free approaches to gain initial access to systems has almost doubled from 40% in 2019, to 75% in 2023. This aligns with claims that over 88% [10] of an adversaries effort is related to gaining initial access to a system, thus bad actors look for lower effort strategies to attack systems. Protecting against vulnerabilities associated with misconfiguration mistakes has been an ongoing challenge for organizations, resulting in the creation of many tools to help with this problem such as Cloud Custodian, a CNCF project that was open sourced by Capitol One.

A 2022 report from the US National Security Agency (NSA) [28] on mitigating cloud vulnerabilities identifies misconfiguration errors in cloud systems as their top concern. A recent 2024 Harvard Business Review (HBR) report [24] highlights the trend of data breaches continuing to increase both in terms of frequency and impact. Specifically the report states that data breaches increased by 20% from 2022 to 2023, but in that same time frame the number of victims doubled. With more than 60% of the world's corporate data estimated to be stored in the cloud, it should not be surprising that in 2023 over 80% of data breaches involved cloud hosted storage.

However, even though cloud providers offer robust security services, and commercial tooling expenditures for security products are estimated to be \$12.8B USD in 2023, data breaches against cloud deployed systems continue to happen at increasing rates. Common issues driving these trends are related to the complexity of managing cloud native environments themselves, and software



This work is licensed under a Creative Commons Attribution 4.0 International License. *AICCC 2024, Tokyo, Japan*

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1792-5/24/12

<https://doi.org/10.1145/3719384.3719463>

architecture choices made in the construction of cloud native applications. Attacks focused on exploiting systems that rely on APIs for integration have resulted in some of the most serious incidents in recent years. These attacks have been successfully executed against some of the most tech-savvy companies in the world including LinkedIn, Facebook, and Microsoft.

One common trend associated with data breach attacks is the amount of time a bad actor is able to access a system undetected. For example, the cyberattack reported by T-Mobile in early 2023 that resulted in the exposure of over 37 million records went undetected for over 40 days. An IBM security report [17] conducted a study with 477 companies, where the mean time to identify a breach was reported as 197 days. The amount of data stolen during data breaches is generally correlated to the amount of time a bad actor can go undetected. Security controls like intrusion protection systems (IPS) need to minimize false positives so they target identifying unusual network connection patterns. The closer an attacker can emulate normal behavior the less likely to be flagged by an IPS, but this also limits the rate of data extraction.

The remainder of this paper explores the detection of stealthy data leaks caused by misconfigured cloud native systems that use API-based integration. This is a popular attack strategy used by cybercriminals because data is exfiltrated using the same APIs as benign users, making detection difficult. We describe a novel tool we created that collects system call data from Linux operating system kernels supporting a Kubernetes [21] cluster. System calls are a good proxy for overall system behavior as they reflect the operations that the kernel is performing on their behalf. We then describe the pre-processing approach that we use to extract features from this data that integrates Kubernetes configuration data with the raw system call information extracted from the kernels in the cluster. Once pre-processed, unsupervised anomaly detection models were trained using this data. Both principal component analysis and autoencoders were used in this study to minimize the reconstruction loss when trained with benign user activity. We then conducted several attacks emulating strategies used by bad actors in other data breaches and investigated if the attack could be detected by exceeding a reconstruction loss threshold.

2 Related Work

Cloud systems can be classified into two broad models: 1) Infrastructure as a Service (IaaS), where software architectures execute on cloud-rented servers provided as virtual machines; and 2) Cloud Native Architectures, where architectures rely on fully-managed cloud services that provide intrinsic support for architectural concerns related to resiliency, auto-scaling, flexible cost management, deployment velocity, data sovereignty and identity management. While many of the security challenges associated with IaaS apply to cloud native systems, this paper specifically focuses on the threats associated with cloud native systems. The architectural benefits of cloud native computing is an active area of study [5, 11, 14, 22, 32]. A recent survey [35] examines many of the unique security challenges associated with protecting cloud native systems. Investigating security threats specific to cloud and cloud-native systems continues to be an active area of research [8, 9, 34] and is covered by publications of leading cybersecurity companies [2, 10, 25].

Using phishing attacks to deploy malware continues to be a popular strategy used by bad actors to infect both traditional and cloud systems. Recent work [6] describes a cloud-based deep learning approach to automate the detection of phishing emails. Once deployed, malware can lead to serious consequences including ransomware [31] attacks, and large data breaches [29]. While once detectable using simple static signature based approaches, modern *metamorphic* and *polymorphic* malware [23] are more stealthy and difficult to detect. Detecting modern malware therefore requires more advanced behavioral detection techniques [4, 16]. The trend of increasing malware sophistication is likely correlated to the time security breaches remain undetected. Kaspersky Labs [20] report an average 5 month detection time, and IBM [17] reports an average of 204 days. After detection, the time to contain and mitigate the breach is critical, which IBM reports an average of 73 days. These long detection times have enabled bad actors to leak vast amounts of data from organizations, and to spread in advance of a ransomware attack. While the focus of this paper is on the detection of cloud native misconfiguration attacks, the behavioral approaches we describe can also be applied to malware-style attacks.

The application of machine learning and advanced AI to combat cybersecurity issues is an active area of research [19, 27]. These AI based systems require a large amount of realistic training data to produce effective models. The dynamic nature of cloud native systems makes them very hard to instrument to collect training datasets that capture their runtime behavior. One promising technology to help with this problem takes advantage of a Linux feature called the extended Berkeley Packet Filter (eBPF) [15]. Most cloud-native platforms are deployed using infrastructure running the Linux operating system. This enables eBPF programs that hook directly into network and system calls kernel events to monitor the runtime behavior of cloud systems. To the best of our knowledge no previous work exists on using eBPF to collect events from the Linux kernel so these data can be used as an accurate proxy of cloud native system behavior targeted at training machine learning models for cloud native anomaly detection. We believe that the creation of our tool, described in Section 4, represents a novel contribution that can be used for future research.

Some related work [13] describes using eBPF-based tooling to improve runtime security monitoring in cloud native systems. Unlike our goal of capturing behavioral runtime data to train ML models, they created a security-focused query language called Secl to define rules that can be matched against real-time data collected in the Linux kernel. Other related works describe using eBPF-based tools on network endpoints such as microservices [26], and using eBPF for performance monitoring in container environments [7]. Other work [1] uses the Linux *strace* to capture a *bag of system calls* representing normal behavior from an executing Docker container. It then emulates attacking the container and looks for differences in the captured system call traces.

3 Attacking Cloud Native Systems

Attacking a secure system is generally a two step process. The first step involves gaining initial access to the system using zero-days, unpatched vulnerabilities, social engineering, or phishing. Once initial access is achieved, adversaries then attempt to move

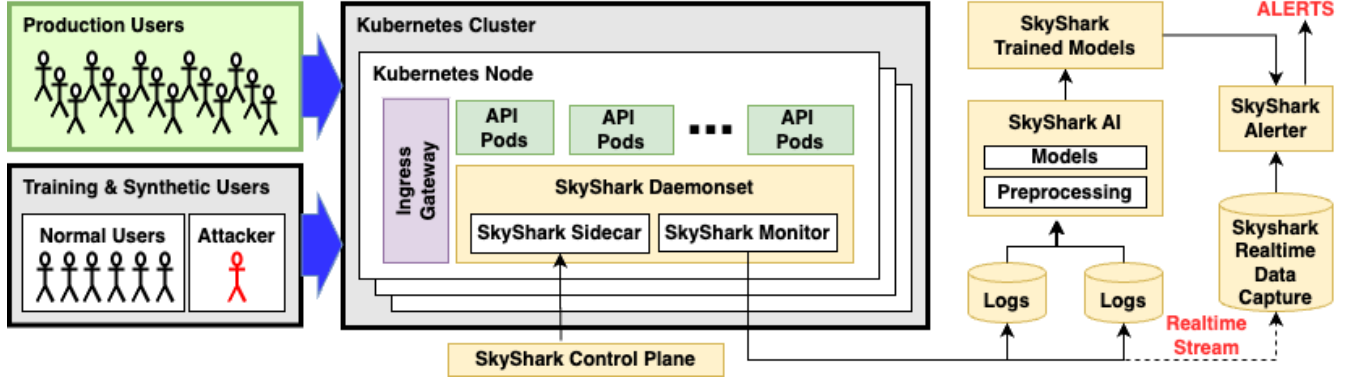


Figure 1: SkyShark Architecture

laterally across the victim’s network, deploying malware to conduct ransomware attacks, or searching for valuable data to exfiltrate.

The effort to gain unauthorized initial access to a cloud native system can be simplified or even eliminated. As companies turn to deploying cloud native systems, the number of publicly facing endpoints accessible over the public Internet increases. In addition to this broader attack surface, initial access is often gained using valid credentials obtained from previous data breaches on the dark web to conduct credential-stuffing attacks. While techniques such as multi-factor authentication (MFA) can help combat these types of attacks, many companies do not deploy them because their customers find using MFA inconvenient. Even when MFA is adopted, large organizations struggle with managing and keeping track of the numerous internet-facing endpoints to ensure MFA is operating as expected. For example, the 2024 ChangeHealthcare ransomware attack, which resulted in a massive disruption to the US Healthcare Industry, has been attributed to the use of stolen credentials to access a remote-access Citrix server that was misconfigured to allow access without using MFA.

Beyond the risks associated with the increase in public endpoints, cloud native systems themselves introduce additional risks. Most modern cloud native systems use RESTful APIs [12] over the encrypted HTTPS transport as a proxy between publicly-exposed applications and the business logic that drives data access to protected databases and internal systems. With the rise of serverless and container orchestration technologies, many organizations now have many hundreds of APIs in their inventory. The adoption and number of deployed APIs is a primary security concern for cloud-native applications. There are many attack vectors that are difficult to protect against including: 1) losing track of publicly-facing APIs; 2) gaining unauthorized access to secured API services via stolen credentials; 3) using misconfigured APIs to gain access to unauthorized data via poorly sanitized or tampered API parameters; 4) assuming API calls used in web and mobile applications cannot be captured and replayed; 5) using enhanced legacy APIs; 6) ensuring that API frameworks are patched; 7) improperly securing API keys and secrets; 8) improperly securing long-lived OAuth refresh tokens that can be exchanged for valid access tokens. This incomplete list of API-specific security concerns enables a new style of attack that is completely malware-free, where the complexity of the system

is exploited to expose software and configuration bugs that enable attackers to gain access.

A recent report [33] highlighted that 94% of respondents to their research study have experienced security problems in production APIs, with 17% having experienced a breach. Another report [3] states that 83% of internet traffic is API traffic. More alarming, these problems cannot be attributed to an immature technical organization, as some of the most notable attacks were successfully executed against leading technical companies. The Open Web Application Security Project (OWASP) recently published [30] a Top-10 list of API security threats that are used to attack cloud native systems. Broadly, these types of vulnerabilities are often referred to as **Cloud Native Misconfiguration Errors**, and will be the focus of our case study described in Section 5.

4 The SkyShark Platform

The architecture of our SkyShark platform, including how it is deployed to monitor Kubernetes-based cloud native architectures to detect cybersecurity issues, is shown in Figure 1. The main component of SkyShark is the Monitor, which is a tool that manages several eBPF filters. The Monitor is designed to instrument any Linux-based runtime environment. Two of the unique features of the Monitor include the ability to activate container- and Kubernetes-specific features when deployed in a cloud environment. Specifically, the Monitor can be configured to intercept container lifecycle activities such as startup and shutdown events from both containerd and docker runtimes. These events enable association of Linux namespaces with containers that we want to observe. The Kubernetes features allow us to map specific APIs to the underlying containers that Kubernetes manages. These features collectively allow us to track and monitor API-specific behavior in a dynamic runtime environment where Kubernetes takes responsibility for autoscaling up and down API instances across a cluster.

The Monitor includes several eBPF filters to intercept system call and network data from the Linux kernel directly. For the purpose of the Case Study (Section 5) we used our SysCallHist monitor, which collects system call data as a proxy to represent the behavior of the cloud native system being observed. For example, we can configure this filter to take a snapshot every second of the total

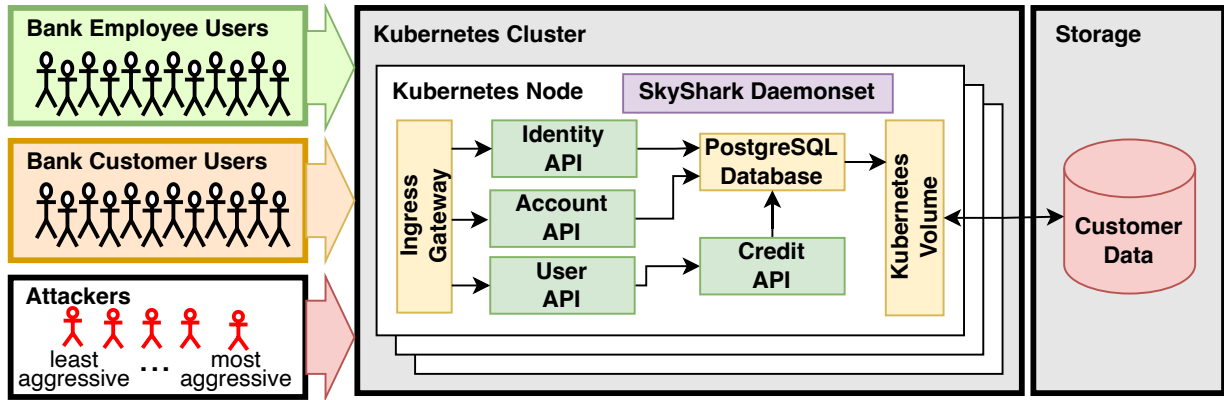


Figure 2: Case Study System Architecture

number of unique system calls executed by individual containers. This data is then logged, where the resulting log file represents an ongoing time-series histogram of the system calls executed for each container in the cloud native system. When enriched with Kubernetes specific metadata, the raw data collected from the kernel can be aggregated to reflect the behavior of individual APIs that are deployed across a cluster.

To integrate with the Kubernetes platform, several additional components were created. First, we let Kubernetes manage the deployment and lifecycle management of our tooling. To do this we use a standard Kubernetes feature called a *daemonset* where Kubernetes ensures one copy of our tool suite is deployed to each node in the Kubernetes cluster. We configure a *pod* that not only includes our monitor, but another component we call the SkyShark Sidecar. Since we want our monitor to be used in many different environments, the SkyShark Sidecar is responsible for starting, stopping and configuring the monitor in real time. Since the Sidecar itself is deployed on every node, we need a way to control all of the sidecars across a cluster. This is where the SkyShark Control plane is used. To configure, start or stop monitoring across the cluster the Control Plane component sends events to the cluster, where each Sidecar maintains a subscription to these events.

The data collection strategy, used to build and train our ML models for anomaly detection, is shown on the right side of Figure 1. A Kubernetes shared volume is used to hold the logs created by the Monitor. Since multiple monitors can be executing across a cluster, each monitor produces its own log file. All of the log records in each file contain the unique machine identifier of the node. To get a time-series view of the behavior across all of the clusters, the log files are aggregated into a single log file and sorted by the timestamps contained in each log record. This data is then transformed into a format suitable to train ML algorithms (Section 6.1). The Monitor can also be configured to emit events over a stream instead of producing a log file for real time analysis.

The final part of the architecture is shown on the left side of Figure 1. We need a realistic way to put a load on a cloud native system to collect data to train our ML models, and to do real-time detection. For training, we use the open source Locust framework,

which allows us to emulate realistic system behavior through the use of synthetic users. For example, to create a benign data, a Locust test was setup to scale-up to 500 concurrent users, where each user executed a random sequence of normal operations. For emulating attacks, we modeled different types of attackers and then setup Locust runs where the system was warmed up with 500 normal users, prior to a single attacker being introduced. The goal of our case study was to see if we could identify attacks in which we varied the attackers aggressiveness based how frequently they made requests, and how much data they were trying to exfiltrate from a misconfigured system.

5 Case Study Environment

The architecture of the cloud native system used in our case study is shown in Figure 2. This system was created to emulate a realistic banking system, where the normal actors include bank employees and customers. This system is deployed on Kubernetes and consists of a number of APIs. Some of the features of this system align with what is expected from a bank including checking account balances, creating new accounts, checking credit scores, and processing credit card transactions. The simulated banking system also incorporates the widely used OAuth system to authenticate users, and to authorize various activities that they can perform through the system’s APIs. We created these APIs so that they can be exploited using common misconfiguration and programming errors found in cloud native systems. Some of the vulnerabilities we enabled include: 1) exploiting API parameters that were designed to be used for internal employees and not customers; 2) inconsistently checking the role of the user; 3) inconsistently sanitizing API parameter inputs enabling authenticated users to request data that they might not be authorized to access; 4) enabling users to obtain valid OAuth access tokens from long-lived refresh tokens without additional challenges or validation.

We used the Locust framework to create simulated users. A total of six user classes were created consisting of a normal user class, and five attacker variations. The attackers exploited the common vulnerabilities we enabled in our banking system to simulate a

realistic data leak attack. The differences between the attackers depend on how aggressively they attack the system to exfiltrate data. We discuss this in Section 7, where we cover our case study in detail. During a realistic data leak attack, bad actors attempt to exfiltrate data by hiding their malicious behavior in the workload created by normal users. All of the attacks in our case study involve loading the system with 500 concurrent normal users and a single attacker. Given the large imbalance between the number of normal users and a single attacker, we wanted to investigate the effectiveness of our anomaly detection techniques as we varied the attackers aggressiveness.

The next section describes our anomaly detection strategy for a data leak attack that exploits common cloud native system misconfiguration errors. It involves converting the data collected by SkyShark into a set of features that can be used to train anomaly detection models using only benign data. We trained a principal component analysis (PCA) model and a deep learning autoencoder. When provided with a snapshots of system call data as input, these models attempt to reproduce the snapshots as output with a minimal reconstruction loss. Our goal was to investigate if these models can produce differentiable reconstruction loss values for inputs that represent normal versus attack scenarios.

6 Anomaly Detection Strategy

Since our case study system is deployed using Kubernetes (K8s) [21] we will start by describing several features of SkyShark that will be used to support our anomaly detection models. Kubernetes manages containerized workloads at runtime by deploying them to one or more pods. Each pod is then scheduled on one or more nodes in a K8s cluster. Kubernetes actively manages the cluster terminating individual pods when no longer needed, restarting pods if they become unhealthy, or adding pods to the cluster to support the load on the system dynamically. As SkyShark collects runtime information from the system it is monitoring, it must also keep track of how K8s is managing the lifecycle of individual pods of interest.

We next focus on how the data collected from SkyShark is used with our anomaly detection models. This data not only includes Linux system call information for our case study system running in a K8s cluster, but also is enriched with K8s metadata by listening for containerd events as Kubernetes manages pods at runtime. Skyshark assumes K8s applications follow best practices and are isolated to specific K8s namespaces. The Skyshark configuration file instructs the monitoring tool which namespaces it should monitor. While eBPF will provide the monitor with all of the syscalls being processed by the kernel, Skyshark will only intercept and log the events that match its configuration. This way individual cloud native system behavior can be isolated.

6.1 Data Collection and Preprocessing

SkyShark uses a JSON format to encode data in the log files it produces. Since SkyShark captures data from a Linux kernel, we end up with individual log files for each node in a K8s cluster. We start by aggregating these files into a master logfile $\mathcal{L}$ by combining all of the per-cluster logfiles and then sorting all records by the timestamp attribute. Next the records in $\mathcal{L}$ are split into distinct

sets based on their type:

$$\begin{aligned}\mathcal{K} &= \{e_i \in \mathcal{L} \mid \text{type}(e_i) = \text{k8s-event}\} \\ \mathcal{W} &= \{e_i \in \mathcal{L} \mid \text{type}(e_i) = \text{syscall-event}\}\end{aligned}$$

The records in $\mathcal{K}$ represent K8s initiated events that SkyShark captured associated with creating and terminating pods in the cluster. The *workload* records in $\mathcal{W}$ are time series data containing data about the system calls being executed on the cluster by the case study system.

We define $\mathcal{K} = \{k_i \in \mathcal{K} \mid k_i = (d_i, p_i, n_i)\}$ as a set of triples containing Kubernetes metadata. In each tuple, d_i represents the Kubernetes deployment name, p_i represents the unique pod name, and n_i represents the Linux PID namespace identifier used to isolate the pod. Because Kubernetes pods are ephemeral, and are isolated from each other using Linux namespaces, $p_i \equiv n_i$. Thus a 1-N map can be defined between d_i and n_i :

$$\mathcal{NS}_{\text{map}} : \forall k_i = (d_i, p_i, n_i) \in \mathcal{K} : n_i \rightarrow d_i$$

The individual log records in $\mathcal{W}$ are a set of triples, where each triple (n_i, sc_i, c_i) records how many times (c_i) a Linux namespace (n_i) executed a specific system call (sc_i). Using $\mathcal{NS}_{\text{map}}$ we consolidate this information by first replacing n_i with d_i , and then summing up all of the c_i for the duplicate (d_i, sc_i) pairs:

$$\begin{aligned}\mathcal{W} &= \{w_i \mid w_i = \{(n_{ij}, sc_{ij}, c_{ij}) \mid j = 1, \dots, |w_i|\}\} \\ &= \{ \{(d_{ij}, sc_{ij}, c_{ij}) \mid (n_{ij}, sc_{ij}, c_{ij}) \in w_i, \\ &\quad d_{ij} = \mathcal{NS}_{\text{map}}(n_{ij})\} \mid w_i \in \mathcal{W} \} \\ &= \left\{ w_i : \{(d_{ij}, sc_{ij}, \sum_{j:(d_i, sc_i)=(d_j, sc_j)} c_j)\}_j \mid \forall w_i \in \mathcal{W} \right\}\end{aligned}$$

Essentially, we consolidate all of the information in $\mathcal{W}$ by transforming the set of triples in w_i from being Linux namespace scoped to representing Kubernetes deployment entities (e.g., individual APIs) deployed across the cluster.

6.2 Feature Extraction

$\mathcal{W}$ is defined as a set of time ordered triples representing the workload of the cloud native system at time step i . For example, let $w_i \in \mathcal{W} = \{(\text{user-api}, \text{accept}, 24), (\text{user-api}, \text{read}, 225), (\text{credit-api}, \text{mmap}, 1050)\}$. This would mean that at time i the user-api invoked the accept system call 24 times and the read system call 225 times. At the same time the credit-api executed 1050 mmap system calls. Note that the ARM64 Linux kernel used in our test environment defines a total of 305 unique system calls that are numbered in the range between 0 and 449. All of the system call data in $\mathcal{W}$ is recorded using the system call number and not its name.

The transformation of the raw system call data into features is best described using an example illustrated in Figure 3. In this example each $D = \{d_1, d_2, \dots, d_n\}$ represents the set of deployed cloud native APIs. System calls are identified by integers and fall in a range $SC \in [0, sc_{\text{max}})$. To keep the the description as simple as possible, Figure 3 represents an example system containing 3 deployments ($|D| = 3$), and supports a simplified hypothetical operating system kernel that only defines 5 system calls ($SC = [0, 4)$):

(a) Raw System Call Data [(deployment_id, system_call_id, exec_count)] at each time step	(b) System Call Arrays for Individual Deployment APIs	(c) Unused System Call Identifiers Removed	(d) Placeholder Column for Testing Added	(e) Features for each time step created by concatenating deployment arrays
t_0 : [(0,1,4),(0,3,9),(1,1,8),(1,3,5),(2,1,6),(2,3,4)] t_1 : [(0,1,2),(1,1,6),(1,3,5),(1,4,4),(2,1,4),(2,4,4)] t_2 : [(0,3,6),(1,1,5),(1,3,7),(2,1,7),(2,3,2),(2,4,6)]	t_0, d_1 : [0,4,0,9,0] d_2 : [0,8,0,5,0] d_3 : [0,6,0,4,0] t_1, d_1 : [0,2,0,0,0] d_2 : [0,6,0,5,4] d_3 : [0,4,0,0,4] t_2, d_1 : [0,0,0,6,0] d_2 : [0,5,0,7,0] d_3 : [0,7,0,2,6]	t_0, d_1 : [4,9,0] d_2 : [8,5,0] d_3 : [6,4,0] t_1, d_1 : [2,0,0] d_2 : [6,5,4] d_3 : [4,0,4] t_2, d_1 : [0,6,0] d_2 : [5,7,0] d_3 : [7,2,6]	t_0, d_1 : [4,9,0,0] d_2 : [8,5,0,0] d_3 : [6,4,0,0] t_1, d_1 : [2,0,0,0] d_2 : [6,5,4,0] d_3 : [4,0,4,0] t_2, d_1 : [0,6,0,0] d_2 : [5,7,0,0] d_3 : [7,2,6,0]	t_0 : [4,9,0,0,8,5,0,0,6,4,0,0] t_1 : [2,0,0,0,6,5,4,0,4,0,4,0] t_2 : [0,6,0,0,5,7,0,0,7,2,6,0] (f) Final Set of Features $\log1p([4,9,0,0,8,5,0,0,6,4,0,0])$ $\log1p([2,0,0,0,6,5,4,0,4,0,4,0])$ $\log1p([0,6,0,0,5,7,0,0,7,2,6,0])$ Removed_SysCalls $R_{sc} = \{0,2\}$

Figure 3: Feature Extraction Process for the Training Dataset

- As input we start with the partially pre-processed dataset $\mathcal{W}$ that was just described for our simplified system.
- A new zero-initialized matrix $S \in \mathbb{R}^{|W| \times |D| \times |SC|}$ is created. System call count data (sc_i) from each $w_i \in \mathcal{W}$ is placed into matrix based on its deployment identifier (d_i) and its timestamp sequence (t_i). For example, the first triple at $t_0 = (0, 1, 4)$ would result in $S[0, 1] = 4$. In general at time t_i , given the triple (d_i, sc_i, c_i) , $S[(t_i * d_i) + d_i, sc_i] = c_i$.
- In this step we drop all of the columns that represent unused system calls, in this case sc_0 and sc_2 . We define a new set R_{sc} to represent the system call identifier that were dropped, in this example $R_{sc} = \{0, 2\}$. This compresses S to help with learning.
- The next step adds another zero column to the end of S . This new column along with R_{sc} from the previous step will be used to support non-training datasets (described below).
- Next S is reshaped, combining adjacent rows for each $d_i \in D$ at time step t_i .
- In the final step we normalize the system call data by applying the $\log1p(x) = \ln(1 + x)$ function to each record to help the ML models train. As mentioned in step (c), we also preserve the R_{sc} set of removed system calls which will be used for non-training datasets.

The process described above is used to create features for training datasets. For non-training datasets the process is slightly different. Step (c) in Figure 3 was used to build R_{sc} , which is the set of syscalls not seen while building the training dataset features. We need a placeholder to store system call counts that might appear in non-training datasets that are in R_{sc} . This is where the zero column in (d) is used. For all non-training datasets, all syscalls in R_{sc} are combined, and their total count is placed into this final column.

6.3 Machine Learning Models

The goal of our case study was to investigate if we could detect a bad actor exploiting common misconfiguration errors associated with API-based cloud native systems. This is a challenging problem because we are not introducing any malware. In malware-based attacks a reasonable assumption can be made that a bad actor exploiting malware is executing different code to attack the system than normal users execute. The goal in this type of attack would be

to see if the ML models could detect the attack by identifying the behavioral differences in the system call patterns when the malware was being executed. In a misconfiguration based attack, errors in the system itself are used as the basis for the attack, thus the system call patterns are likely to closely resemble normal behavior. As a additional complexity, the attacker will be present while the system is being used by many benign users, enabling the bad actor to hide inside of the behavior generated by normal user traffic.

Common unsupervised anomaly detection approaches, where a massive imbalance exists between normal and malicious behavior in datasets, is to use reversible dimensionality reduction techniques. These approaches only require benign data for training. The idea is to train the models so that in the first step they significantly reduce the dimensionality of the input set of features. In the second step they reverse the process in an attempt to reproduce the original set of features from the output of the first step. A *reconstruction loss* function is used to measure the similarity of the output from the second step with the original set of features used as input. Consistently achieving a small reconstruction loss would be an indicator that the model learned how to reconstruct the original features representing normal behavior from a compressed representation. When used for anomaly detection, an effective model after training would produce a small reconstruction loss when presented with data representing normal behavior, and a larger reconstruction loss when presented with data when the system was being exploited. We measure reconstruction loss $RL = (y_i - \hat{y}_i)^2$ by calculating the squared error between an input y_i and output $\hat{y}_i$ set of features predicted by the ML algorithm.

We implemented two different reversible dimensionality reduction algorithms to see how well they can perform anomaly detection. The first algorithm trained a *Principal Component Analysis* (PCA) model, and the second one involved creating a deep *autoencoder* network. The PCA model we used was supplied by the *scikitlearn* python library, and our custom autoencoder was creating using the *pytorch* deep learning framework.

6.4 Building the Training Dataset

We train dimensionality reduction ML models only on benign data. For the case study the training dataset was created by having SkyShark monitor the simulated banking system for 60 minutes.

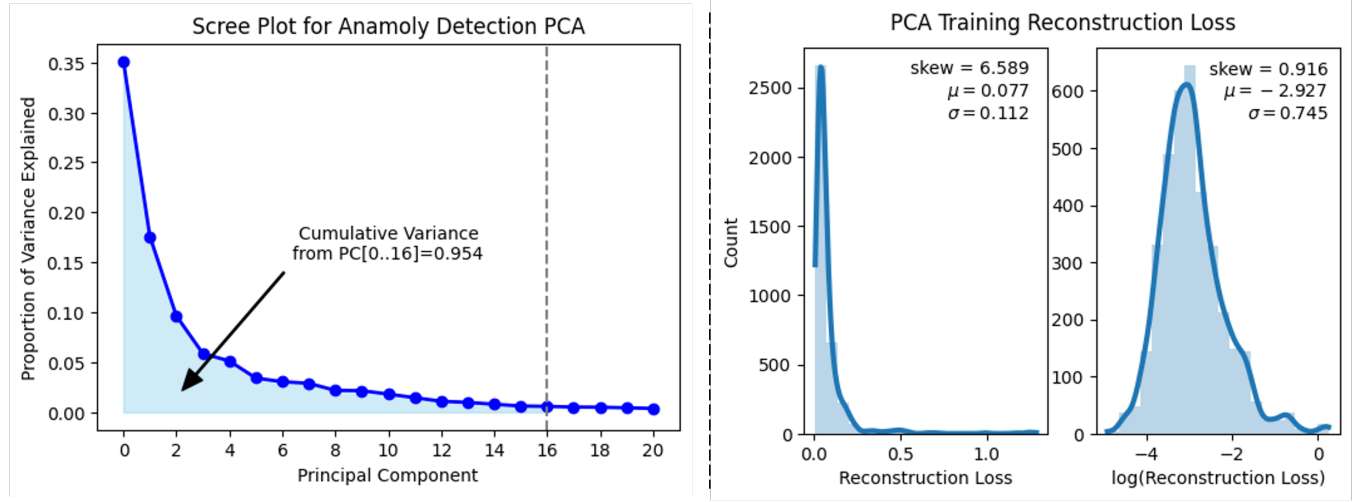


Figure 4: Principal Component Analysis Model

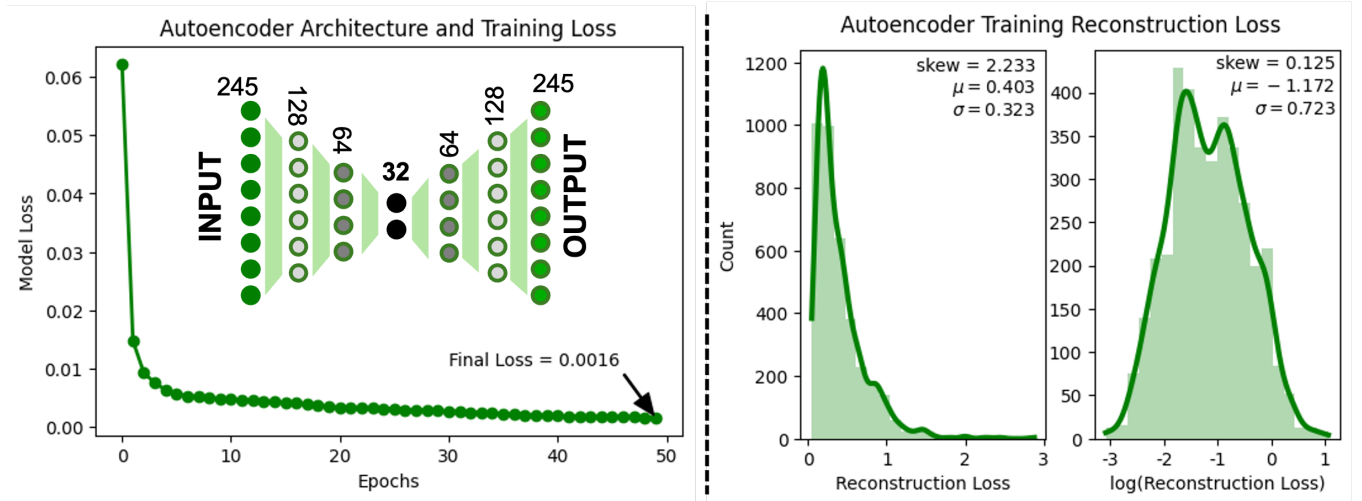


Figure 5: Autoencoder Model

Table 1: Training and Attack Experimental Models

Dataset	Runtime	Normal Users	Attackers	Attacker Data Leak per Request
Training	60 minutes	500 concurrent	0	None
Validation	15 minutes	500 concurrent	0	None
Leak Attacker	15 minutes	500 concurrent	1	Between 1 and 25 records
Slow Attacker	15 minutes	500 concurrent	1	Between 1 and 250 records
Moderate Attacker	15 minutes	500 concurrent	1	Between 250 and 500 records
Aggressive Attacker	15 minutes	500 concurrent	1	Between 500 and 1000 records

We put a simulated load on the system with 500 benign users concurrently calling the system APIs, with a randomized wait time between API calls that ranged from 1 to 5 seconds to introduce some realistic jitter. The shape of the training dataset was [3600, 245]. A total of 3600 records were captured, representing one record per second. Each row consisted of 245 features calculated using the process described in Section 6.2. Skyshark intercepted 73.9 million

system calls while monitoring our test system containing 4 APIs and a database pod.

6.5 Training the Principal Component Analysis Model

PCA works by taking an input set of features and finding a set of orthogonal linear combinations of these features, called principal components. These new principal component features are uncorrelated and sorted by the amount of variance they explain in the input data. Fitting a PCA model to the input data often leads to the first few principal components covering most of the variance in the dataset.

On the left side of Figure 4 we show a *Scree Plot* of the first 20 principal components after fitting the PCA model to the training dataset. The first 16 components cover over 95% of the variance in the training data. The right side of Figure 4 shows the reconstruction loss for the training data set. The model performs very well producing a mean loss of $\mu = 0.077$. In order to perform anomaly detection, we also need to determine a reconstruction loss threshold such that a value above this threshold would indicate anomalous behavior. Ideally this threshold would minimize both false positives and false negatives. The 6.589 skew of the reconstruction loss shows that the data likely follows a lognormal distribution (which can also be inspected by looking at Figure 4). To correct for this, we take the natural logarithm of the loss and end up with a much better skew, 0.916, which indicates the data is approximately normally distributed. Next we calculate the 99.5th percentile which was -0.664 . Finally, we establish the threshold by transforming the value back to the lognormal space, setting the RL threshold to $e^{-0.664} = 0.51$.

6.6 Autoencoder Model

An autoencoder is a deep learning network that consists of an encoder stage that produces output that is fed into a decoder stage. The encoder compresses the input data into a latent space representation through a series of layers, each applying linear transformations and non-linear activation functions. The decoder then reconstructs the original input data from the compressed representation, mirroring the encoder’s architecture in reverse. Together, these components form a neural network that learns to efficiently encode and decode the input data to minimize reconstruction loss.

On the left side of Figure 4 we show the architecture of the autoencoder we created for the case study, along with the model loss during training. We trained the autoencoder for 50 epochs, with a batch size of 32, using the MSE loss function and the Adam optimizer configured with a learning rate of 0.001. The input and output units of the autoencoder matched the size of the number of features in our training dataset. The architecture of our deep autoencoder is also shown on the left side of Figure 4. Each encoder phase progressively reduces the dimensionality of the input data – in our 3 layer network we go from $245 \rightarrow 128 \rightarrow 64 \rightarrow 32$. The latent layer in the center represents both the output of the encoder and the input to the decoder. The decoder builds up to the output layer using dimensionality expansion layers that mirror the encoder. In our network the encoder part goes from $32 \rightarrow 64 \rightarrow 128 \rightarrow 245$.

After the autoencoder was trained, we fed the training data through the model and measured the reconstruction loss. These results are shown on the right side of Figure 4. The results were very good, but the distribution was also lognormal. We followed the same process as we did with the PCA to calculate the anomaly threshold

at the 99.5th percentile. For the autoencoder the RL threshold was established at $e^{0.398} = 1.49$.

7 Case Study Execution and Results

Table 1 defines the parameters used to setup our training, validation, and attack models for the case study. All experiments were conducted using a load of 500 concurrent users performing normal operations. The *Training* run was executed for 60 minutes with no attackers for the purpose of training our PCA and autoencoder models. The *Validation Test* was run for 15 minutes to measure the quality of the model training process. The last 4 experiments introduced a single attacker, with the only difference being the “aggressiveness” of how much data they were leaking per request. Since most data leak attacks are reported in terms of the number of records stolen, we follow this approach in the case study, varying how much each attacker is attempting to steal per request. To make the attack more realistic and difficult to detect, the simulated attackers select a random number of records to steal on each request from within the range described in Table 1. For both the PCA and autoencoder models we explored three different attack rates:

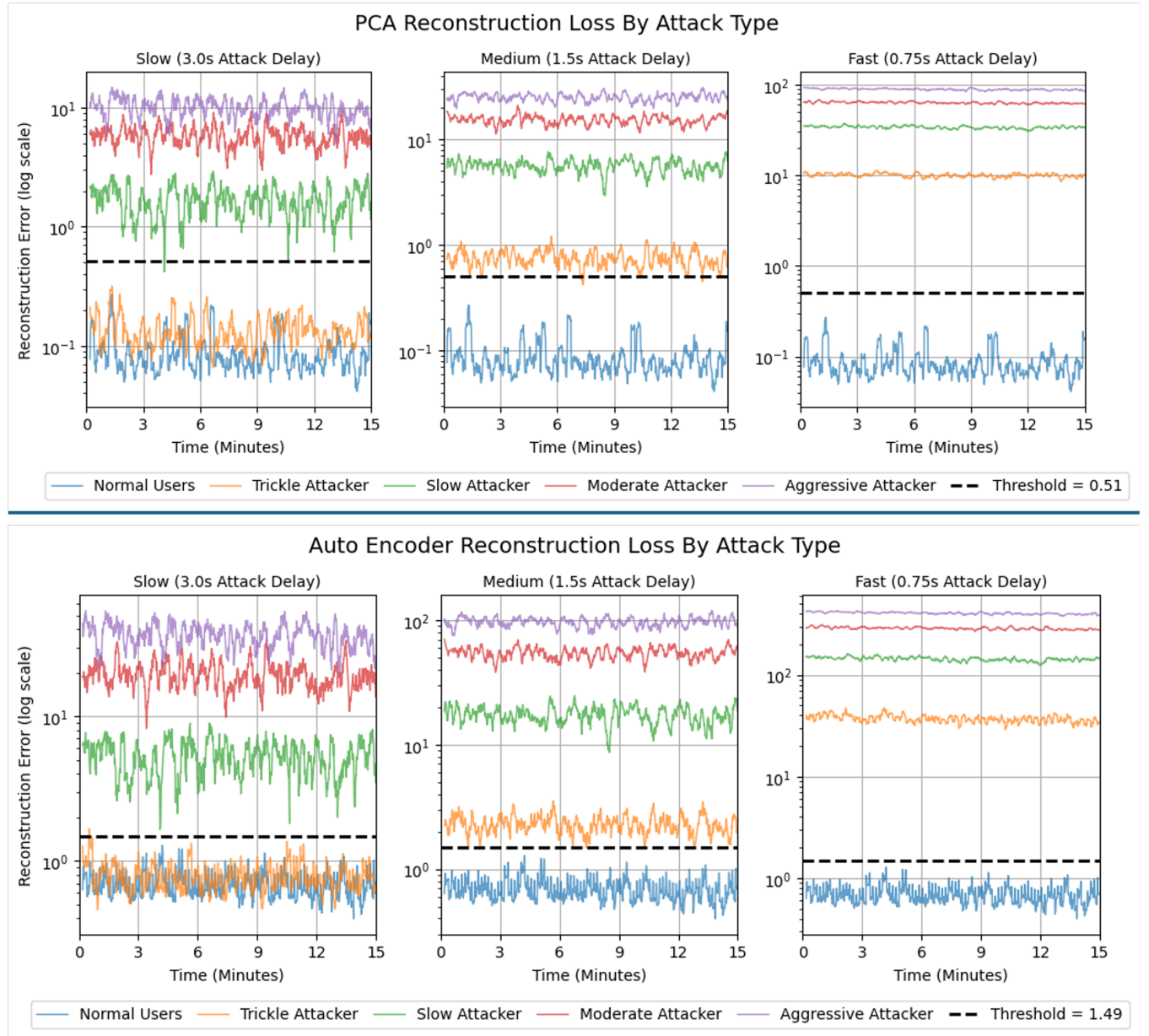
- **Fast Attack Set:** random delay between requests in the range of 0.5 to 1.0 seconds, with the average being 0.75 seconds.
- **Medium Attack Set:** random delay between requests in the range of 1.0 to 2.0 seconds, with the average being 1.5 seconds.
- **Slow Attack Set:** random delay between requests in the range of 1.0 to 5.0 seconds, with the average being 3 seconds. Note this was also the same random request rate used to simulate normal benign users.

7.1 Results

The case study involved collecting a total of 14 datasets. We collected one dataset for training, and one dataset for validation of our trained models. The remaining 12 datasets were collected by running the *Fast*, *Medium*, and *Slow* attack sets using each of our 4 attacker personas, for a total of 12 datasets.

The results from our Case Study are shown in Figure 6. Both the PCA and autoencoder models did a good job identifying the presence of the different attacker classes. Both models produced zero false positives when only normal benign users were present. This is not surprising given that we can set the RL threshold to whatever value we want to control the false positive rate (FPR). By biasing our results to lower the FPR, we risk trading off a larger false negative rate (FNR). The ideal outcome is to have a zero FPR and FNR, meaning the models classified true positives and true negatives 100% of the time. Table 2 summarizes the case study results. Our models had a difficult time classifying when a *trickle attacker* was performing a *slow attack*. This is likely caused by the attacker closely mirroring the behavior of a normal user to the point where they cannot be detected.

While our system did a very good job detecting anomalous behavior for all cases except for when a *trickle attacker* was performing a *slow attack*, this outlier case requires some perspective. Many notable data leak attacks over the past year resulted in over 50M stolen records. To steal that much data with the single case we

**Figure 6: Anomaly Detection Reconstruction Loss Results**

failed to detect would require the attacker to be constantly running for almost 139 days. The attacker would need to exhibit extreme patience over this timeframe. Also, network conditions are likely to change over a 139 day timeframe where the possibility of detecting the attacker could improve.

8 Conclusions

In this paper we focused on attacks resulting from misconfiguration errors in cloud native systems. These types of attacks have been increasing due to the lower sophistication required to attack the complexity of the system itself. We also discussed a novel

tooling platform, called SkyShark, we created to monitor cloud native systems that uses eBPF technology to capture high-fidelity system-specific data that can be used for anomaly detection without requiring any source code modifications. Unlike other approaches that seek generalized solutions, we believe that better results can be obtained by targeting system-specific behavior coupled with simple ML models that are fast to train and simple to integrate into existing CI/CD pipelines. We discussed how the raw data used by SkyShark can be transformed into features used to train unsupervised ML models including a PCA and an autoencoder. A case study was conducted that investigated if various types of attackers, executing

Table 2: Case Study Results (Sorted by F1 Score)

ML Model	Attacker Type	Attack Type	Precision	Recall	F1 Score
PCA	Trickle	Slow	0.000	0.000	0.000
AE	Trickle	Slow	1.000	0.024	0.048
PCA	Slow	Medium	1.000	0.967	0.983
PCA	Slow	Slow	1.000	0.997	0.998
All Others	*****	*****	1.000	1.000	1.000

attacks at different rates, could be detected while the system was concurrently loaded with normal users. This problem is challenging because the attacker can use the normal user traffic to hide their malicious behavior. As future work we plan to revisit this problem and incorporate some new network sensors we recently added to the SkyShark platform.

Acknowledgments

This research was funded by the Auerbach Berger Chair of Cyber-security held by Spiros Mancoridis.

References

- [1] Amr S. Abed, T. Charles Clancy, and David S. Levy. 2015. Applying Bag of System Calls for Anomalous Behavior Detection of Applications in Linux Containers. In *2015 IEEE Globecom Workshops (GC Wkshps)*. 1–5. doi:10.1109/GLOCOMW.2015.7414047
- [2] Akamai. 2024. *Lurking in the Shadows: Attack Trends Shine Light on API Threats*. <https://www.akamai.com/lp/soti/lurking-in-the-shadows> Accessed: March 4, 2024.
- [3] Akamai. 2024. *State of the Internet*. <https://www.akamai.com/security-research/the-state-of-the-internet> Accessed: March 4, 2024.
- [4] Ömer Aslan Aslan and Refik Samet. 2020. A Comprehensive Review on Malware Detection Approaches. *IEEE Access* 8 (2020), 6249–6271. doi:10.1109/ACCESS.2019.2963724
- [5] Dhanyaa Bharadwaj and B. S. Premananda. 2022. Transition of Cloud Computing from Traditional Applications to the Cloud Native Approach. In *2022 IEEE North Karnataka Subsection Flagship International Conference (NKCon)*. 1–8. doi:10.1109/NKCon56289.2022.10126871
- [6] Umer Ahmed Butt, Rashid Amin, Hamza Aldabbas, Senthilkumar Mohan, Bader Aloufi, and Ali Ahmadian. 2023. Cloud-based email phishing attack using machine and deep learning algorithm. *Complex & Intelligent Systems* 9, 3 (2023), 3043–3070.
- [7] Cyril Cassagnes, Lucian Trestioreanu, Clement Joly, and Radu State. 2020. The rise of eBPF for non-intrusive performance monitoring. In *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 1–7.
- [8] Milan Chauhan and Stavros Shiaeles. 2023. An analysis of cloud security frameworks, problems and proposed solutions. *Network* 3, 3 (2023), 422–450.
- [9] Maxim Chernyshev, Zubair Baig, and Sherali Zeadally. 2021. Cloud-Native Application Security: Risks, Opportunities, and Challenges in Securing the Evolving Attack Surface. *Computer* 54, 11 (2021), 47–57.
- [10] CrowdStrike. 2024. *2024 Global Threat Report*. <https://www.crowdstrike.com/global-threat-report/>
- [11] Shuiguang Deng, Hailiang Zhao, Binbin Huang, et al. 2024. Cloud-Native Computing: A Survey From the Perspective of Services. *Proc. IEEE* 112, 1 (2024), 12–46. doi:10.1109/JPROC.2024.3353855
- [12] Roy T Fielding and Richard N Taylor. 2002. Principled design of the modern web architecture. *ACM Transactions on Internet Technology (TOIT)* 2, 2 (2002), 115–150.
- [13] Guillaume Fournier, Sylvain Afchain, and Sylvain Baubeau. 2021. Runtime security monitoring with ebpf. In *Proceedings of the 17th SSTIC Symposium sur la Sécurité des Technologies de l'Information et de la Communication*. 2–4.
- [14] Dennis Gannon, Roger Barga, and Neel Sundaresan. 2017. Cloud-Native Applications. *IEEE Cloud Computing* 4, 5 (2017), 16–21. doi:10.1109/MCC.2017.4250939
- [15] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A Navas, Noam Rinetzk, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 1069–1084.
- [16] Deepak Gupta and Rinkle Rani. 2020. Improving malware detection using big data and ensemble learning. *Computers & Electrical Engineering* 86 (2020), 106729. doi:10.1016/j.compeleceng.2020.106729
- [17] IBM Security. 2018. *IBM Study: Hidden Costs of Data Breaches Increase Expenses for Businesses*. IBM. <https://newsroom.ibm.com/IBM-security?item=30567>
- [18] IBM Security. 2023. *Cost of a Data Breach Report 2023*. <https://www.ibm.com/reports/data-breach> Accessed: 3/4/2024.
- [19] Jain Jaya. 2022. Application of Deep Learning in Cloud Security. *Deep Learning Approaches to Cloud Security* (2022), 189–205.
- [20] kaspersky. 2023. *How Data Breaches Happen*. <https://usa.kaspersky.com/resource-center/definitions/data-breach> Accessed: March 4, 2024.
- [21] Kubernetes. 2023. *Kubernetes (k8s)*. Retrieved January 3, 2023 from <https://kubernetes.io>
- [22] Frank Leymann, Uwe Breitenbücher, Sebastian Wagner, and Johannes Wettinger. 2017. Native cloud applications: why monolithic virtualization is not their foundation. In *Cloud Computing and Services Science: 6th International Conference, CLOSER 2016, Rome, Italy, April 23–25, 2016, Revised Selected Papers* 6. Springer, 16–40.
- [23] Xufang Li, Peter K.K. Loh, and Freddy Tan. 2011. Mechanisms of Polymorphic and Metamorphic Viruses. In *2011 European Intelligence and Security Informatics Conference*. 149–154. doi:10.1109/EISIC.2011.77
- [24] Stuart Madnick. 2024. *Why Data Breaches Spiked in 2023*. Harvard Business Review. <https://hbr.org/2024/02/why-data-breaches-spiked-in-2023>
- [25] Mandiant. 2022. *Mandiant Cyber Security Forecast 2023*. <https://www.mandiant.com/resources/reports/mandiant-cyber-security-forecast-2023> Accessed: March 4, 2024.
- [26] Sebastiano Miano, Fulvio Rizzo, Mauricio Vásquez Bernal, Matteo Bertrone, and Yunsong Lu. 2021. A framework for eBPF-based network functions in an era of microservices. *IEEE Transactions on Network and Service Management* 18, 1 (2021), 133–151.
- [27] Ali Bou Nassif, Manar Abu Talib, Qassim Nasir, Halah Albadani, and Fatima Mohamad Dakalbab. 2021. Machine learning for cloud security: a systematic review. *IEEE Access* 9 (2021), 20717–20735.
- [28] National Security Agency (NSA). 2023. *Mitigating Cloud Vulnerabilities*. https://media.defense.gov/2020/Jan/22/2002237484/-1-1/0/CSI-MITIGATING-CLOUD-VULNERABILITIES_20200121.PDF Accessed: March 4, 2024.
- [29] NIST. 2023. *Progress MOVEit Transfer SQL Injection Vulnerability*. Retrieved June 1, 2024 from <https://nvd.nist.gov/vuln/detail/CVE-2023-34362>
- [30] OWASP. 2023. *OWASP API Security Project*. <https://owasp.org/www-project-api-security/> Accessed: March 4, 2024.
- [31] Harun Oz, Ahmet Aris, Albert Levi, and A Selcuk Uluagac. 2022. A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–37.
- [32] Dimpi Rani and Rajiv Kumar Ranjan. 2014. A comparative study of SaaS, PaaS and IaaS in cloud computing. *International Journal of Advanced Research in Computer Science and Software Engineering* 4, 6 (2014).
- [33] Salt Security. 2023. *State of API Security Report Q1 2023*. Retrieved June 1, 2024 from <https://content.salt.security/state-api-report>
- [34] Hamed Tabrizchi and Marjan Kuchaki Rafsanjani. 2020. A survey on security challenges in cloud computing: issues, threats, and solutions. *The journal of supercomputing* 76, 12 (2020), 9493–9532.
- [35] Theodoros Theodoropoulos, Luis Rosa, Chafika Benzaid, et al. 2023. Security in Cloud-Native Services: A Survey. *Journal of Cybersecurity and Privacy* 3, 4 (2023), 758–793.